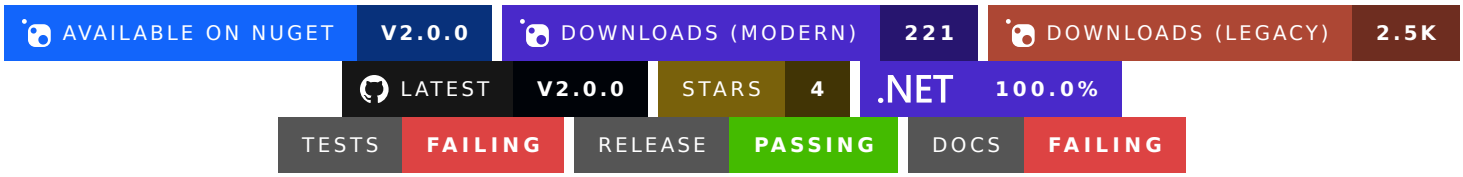
Red Sea Markup Language



An OceanApocalypseStudios project,
the modern language designed to dynamically interpret different logic paths based on an host's OS and CPU architecture.

Why RSML?

Red Sea Markup Language is **the** powerful and robust fork of MF's CrossRoad Solution, a language designed to **dynamically interpret different logic paths based on the local host OS and CPU architecture**.

RSML, which is short for Red Sea Markup Language, is still in development, but the finished v3.0.0 will observe the following core features:

- A complete toolchain featuring buffers, readers, lexers, parsers and evaluators.
- The core library available in C#, Visual Basic, Python and many other languages.
- SDK for convenience and a more idiomatic usage of RSML in C#.
- A native shared library for those using RSML in C, C++ or languages that don't have official bindings.
- A CLI for interacting with the RSML toolchain easily.
- A static website for evaluating RSML on the go.
- First-class support for Visual Studio.
- Syntax highlighting for Visual Studio Code, JetBrains IDEs, Notepad++ and more.

RSML solves the issue of resolving logic paths dynamically based on a given host's characteristics. When assigned an host, which could be the local host, RSML solves the logic paths and returns the first match's associated value. The important part here is to note RSML does this dynamically: if you were to switch hosts or pass different data, RSML would adapt accordingly.

We :heart: open-source!

v3.0.0-prerelease1

🚀 Features

- *(api)* [**breaking**] Add the extensibility project
- *(buffer)* [**breaking**] Implement the read-only span buffer for minimum allocations
- *(toolchain)* Implement all members from `IToolchainComponent` in the classes that implement it
- *(buffer)* Remove `GetWord`-like methods and implement `GetSourceSpan` in the read-only string buffer
- *(buffer)* Add more equality checks for the buffers
- *(extensions)* Start working on extension support little by little
- *(buffer)* Add properties for checking if a source is read-only and checking its cursor position
- [**breaking**] Add multi-target support for .NET 8.0, .NET 472 and .NET 481
- *(buffer)* [**breaking**] Add array-based methods to the read-only buffer interface and class, for convenience (#53 closed by #59)
- *(buffer)* Add methods for calculating the length of a line in the buffer (#58)
- *(buffer)* Add and test `TryGetLineFromIndex` method to the read-only string buffer (#57)
- Featuring nuget trends

🐛 Bug Fixes

- [**breaking**] Fixed major bug where `SourceSpan` did not check if the start index was less than the end index
- *(buffer)* Fix last line specific logic when counting until the end of the line
- *(buffer)* Fix out of range calculations not taking negative indexes into account
- *(buffer)* [**breaking**] Fix `CountUntilLineSeparator` returning off-by-one and negative results
- *(buffer)* Made a condition more obvious on what the consequences are
- *(test)* Fix MTP setup
- *(buffer)* Fix methods and line counting mechanics in the read-only string buffer
- Fix and test native exports (1/?)
- Fix maybe??

🚧 Refactor

- Place all internal-use extension members in a single file
- Suppress minor issues and remove unnecessary extension methods
- *(buffer)* Rename the read-only string buffer to `ReadOnlyCharBuffer` to indicate `TItem` is `System.Char` instead of `System.String`
- *(buffer)* Add `GetSourceSpan` method and remove variations of `GetWord`

- Rename `CountUntilLineSeparator` to `CountUntilEndOfLine`
- Rename `InternalUtils` to `Constants` and move the OAS item interface to a better namespace
- Use C# 14 extension blocks instead of static extension methods
- *(sdk)* Remove `IInjectable` for not being necessary anymore
- *(sdk)* Reorganized the RSML API and SDK
- Move types around namespaces and add directories to have code in the future
- *(buffer)* Move the buffers to the parent `Sources` directory (and namespace)
- *(cache)* Move `ISupportsCache` to a dedicated cache directory (and namespace)
- *(buffer)* Remove unnecessary methods and remove `Try*` pattern from `TryGetSourceLocation`

⚡ Performance

- *(buffer)* [**breaking**] Use result value types over exceptions for performance
- Perform a solution-wise code cleanup
- Perform solution-wise code cleanup and test the native normalizer (#29)

📚 Documentation

- *(buffer)* Document exceptions thrown by methods and EOF conventions
- *(buffer)* Document behavior where EOF convention is not followed by location-finding method
- *(api)* Update .NET API documentation
- *(blog)* Add article explaining the amount of tests implemented
- *(buffer)* Document `GetLine`'s EOF conventions
- *(buffer)* Add notes about EOF conventions in XML documentation of some methods
- *(blog)* Add the very first blog article and organize documentation better
- Mark documentation as to be done
- Migrate from mkdocs to DocFX
- Remove my public email address from the RSML docs
- Improve documentation partially
- Fix broken shortcodes
- Docs have been moved here

🎨 Styling

- *(docs)* Improved light theme for the documentation website

🧪 Testing

- *(buffer)* Finish testing the read-only string buffer and fix some faulty behaviors while computing line starts

- *(buffer)* Test constructors and slicing methods for the read-only string buffer
- *(buffer)* Implement tests for `GetLineFromIndex` and `GetSourceLocation`
- *(buffer)* Test error-throwing buffer logic
- *(buffers)* Test fixed version of `CountUntilEndOfLine` and add more tests for it
- *(buffer)* Add tests for verifying if `GetLengthOfLine` thrown when empty or out of range
- *(buffer)* Add tests for equality checks within the read-only string buffer
- *(buffer)* Test more counting functions and further test `CountWhile`
- *(buffer)* Heavily test several buffer functions related to counting
- [**breaking**] Add test projects for testing RSML, RSML's SDK and RSML's Native exports
- Test `CountLinesBefore` method and close #51
- Test the native evaluator (#29) and improve native <-> managed conversions
- Test the native validator as requested in #29

⚙️ Miscellaneous

- Include new extensibility project to security information
- *(deps.nuget)* Bump the minor-and-patch group with 3 updates (#66)
- Reduce GitHub Actions usage time by running automated tests only on .NET 10.0
- Update git attributes with new content and normalize file encodings
- Update changelogs with latest commits
- Remove useless comments and update security policy
- Remove SDK project and have the SDK in the RSML project instead
- Update changelog with latest available data
- Update release workflow to use Trusted Publishers over API keys
- Setup code owners for the repository
- Fix Dependabot workflow issue
- Update workflows and add security information
- Remove docs output from the repository
- Add platform support to the RSML solution
- Make the CLI non-packable
- Debug release workflow on Linux ARM64
- Debug release workflow on macOS
- Improve README.md and add test badges
- Prefer bash over PowerShell to avoid YAML escaping issues
- Update workflow to avoid linker issues
- Fix workflow toa void parser error on Windows ARM64
- Make RSML.Native not pack on build
- Comment out the release step (the only goal is testing)
- Move the MTP migration property to a props file
- Migrate to MTP
- Fix and update workflows

- Remove old benchmarking results

> **NOTE**
> Previous to `v3.0.0-prerelease1`, Conventional Commits were not enforced in the repository, hence the lack of content below this note.

2.0.0 @ 2025-09-09

🚧 Refactor

- Refactoringsss

⚡ Performance

- Its actually good now

2.0.0-prerelease8 @ 2025-08-18

⚡ Performance

- PERF. OR. MANCE.
- Performance goes boom

📚 Documentation

- Docs (part 3) not much done tho
- Docs (part 2)

🧪 Testing

- Tests not passing life sux

1.0.1 @ 2025-05-31

📚 Documentation

- Docs part 1

1.0.0 @ 2025-05-28

🐛 Bug Fixes

- Fixed it
- Fixed it (hopefully)

TODO

Namespace OceanApocalypseStudios.RSML

Namespaces

OceanApocalypseStudios.RSML.Cache

OceanApocalypseStudios.RSML.Diagnostics

OceanApocalypseStudios.RSML.Execution

OceanApocalypseStudios.RSML.Extensibility

OceanApocalypseStudios.RSML.Language

OceanApocalypseStudios.RSML.Native

OceanApocalypseStudios.RSML.Sources

Interfaces

IToolchainComponent
: A component of the RSML toolchain.

Enums

ToolchainConfiguration
: Configuration options for a IToolchainComponent.

List of Supported Programming Languages

RSML v3.0.0 will support native interop by exporting a C ABI, meaning it can be used in any language that supports C interop.

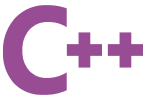
However, to make it easier for people who are unexperienced with C, we plan on creating packages for popular languages so people can use high-level APIs instead of having to struggle with DLLs.

Main Releases

Languages	Is available?	Documentation	Is officially maintained?	Official Download Link	Repository	I
C#	🟡 As a pre-release.	🔴 Out-of-date *(new documentation coming soon)*	🟢			S c
	🟡 As a pre-release.	🟣 No specific documentation planned (refer to **RSML for C#** docs).	🟢			S c
VB	🟡 As a pre-release.	🟡 Partial documentation only (coming soon).	🟢			S c

Other Releases

Languages	Is available?	Documentation	Is officially maintained?	Official Download Link	Repository	I
C	🟡 DLL available as a pre-release. Header file	🔴 Coming soon.	🟢 Will be available as an official OAS project.	🔴 Coming soon.	🔴 Coming soon.	-

	coming soon.					
	🔴 Coming soon.	🔴 Coming soon.	🟢 Will be available as an official OAS project.	🔴 Coming soon.	🔴 Coming soon.	-
	🔴	---	---	---	---	-
	🔴	---	---	---	---	-
	🔴	---	---	---	---	-
	🔴	---	---	---	---	-
	🔴 New version coming soon.	🔴 Out-of-date *(new documentation coming soon)*	🟢			(s
	🔴	---	---	---	---	-
	🔴	---	---	---	---	-

TODO

TODO

Available Guides and Walkthroughs

Why 400 tests?

> Written by Matthew at OceanApocalypseStudios on July 14, 2026.
>
> It will take about 1:45 to read this article at 220 WPM *(average reading speed)*.

Before we get into the article, I want to inform you it will be much shorter than the last one, mostly because, currently, the focus is finishing #60 and finally moving on to RSML's lexer and respective modularity system.

Today, we have officially hit the following milestone in RSML: 400 test methods... for a single class... and it's still not fully tested. Before you bring your anti-testing and `Console.WriteLine()` pitchforks forward, let us explain why we have implemented that many test methods...

Context matters

In this case, context matters a lot. The class that has received this much testing attention is `ReadOnlyStringBuffer`, and it serves as the simplest buffer available in the entirety of the official RSML API, yet 400 tests were only half* of what was needed to get it to work properly.

At OceanApocalypseStudios, we believe testing buffers and big language components (such as lexers and parsers) is imperative to ease the task of getting them to work correctly. If this means testing a lot, then it's a sacrifice that we believe is worth making.

It's worth mentioning this does **not** mean in any way that we believe testing is only relevant when the components are critical: testing is still important when they're not — the only difference is that it's even more important when the components are critical.

Following the same rationale, we are **not** saying quantity matters more than quality: RSML could have 9000 tests and could be a broken mess or have 10 tests and work perfectly. We believe a good combination of both is the best compromise a developer can choose.

*This is an assumption, based on the current amount of test methods and the amount of methods that are yet to be tested.

Expecting more tests?

One word: **yes**. Not because we love writing tests, but because there are still methods that are publicly exposed yet are fully untested by now. Hopefully, they won't stay that way for much longer, and I'll be happy to write a new article once we can finally mark #60 as closed.

Like I said, this was quite a small article, and a bigger one is to be expected soon.

Thank you for dropping by and reading! ❤️